ICDeC — LEN

INTERNSHIP WORKBOOK

Weekly Engineering Review (WER)

Revised Edition — Book-Grade Format

Sistem Kontrol Dispenser Berbasis Perintah Suara Pada STM32U5A9J-DK: Studi Kinerja Dsp Extension Arm Cortex-M33 Dalam Pemrosesan Audio

PURPOSE OF THIS DOCUMENT

This Weekly Engineering Review (WER) is not merely a logbook or progress report. It is a systematic engineering knowledge document designed to:

- Preserve the full learning history, design rationale, and verification evidence of each week's engineering work.
- Record design decisions with alternatives considered and technical justification — forming the methodology section of a future academic book.
- Capture failures and root causes explicitly — failures are as academically valuable as successes.
- Generate reusable technical guidance for future student cohorts (institutional memory).
- Accumulate peer-reviewed literature references week by week, forming the book's bibliography organically.
- Produce publishable-quality chapter material each week by disciplined documentation of experiments, measurements, and insights.

Every entry must answer: What was designed? What was proven? What failed and why? What should the next student know?

SECTION DC DOCUMENT CONTROL

Complete all fields before submitting the WER. Incomplete headers will be returned for revision.

Perancangan Sistem Kontrol Dispenser Berbasis Perintah Suara Pada STM32U5A9J-DK: Studi Kinerja Dsp Extension Arm Cortex-M33 Dalam Pemrosesan Audio

Student Name	*Gregory Salman Ahmad*
Student ID / NIM	*13223016*
Industrial Mentor	*Abdillah Aziz*
Academic Supervisor	-
Week Number	*W1*
Reporting Period	*30 June 2026 – 06 July 2026*
Repository / Folder Link	github.com/Gregoreuy/ICDEC-LEN-KP-DSP-Evaluation
Report Version	*Rev. 0.1 = draft submitted to mentor; Rev. 1.0 = mentor-approved final*
Submission Date	*07-07-2026*

SECTION 1 WEEKLY OBJECTIVE

This week covers the completion of the I2S/SAI1 + GPDMA audio acquisition baseline (WBS 2.2, 3.1–3.3) together with the end-to-end integration of a pre-trained Edge Impulse “Yes-No” keyword-spotting (KWS) model, pulled forward ahead of the original 8-week schedule (WBS 4.1–4.2, originally planned for Week 2).

Item	Description — Write Your Entry Below
Main Weekly Objective	Finalize and validate the INMP441 I2S audio acquisition pipeline (SAI1 Master Receive + GPDMA1 circular DMA) on the STM32U5A9J-DK, and integrate a pre-trained Edge Impulse “Yes-No” KWS classifier end-to-end — from static offline test to real-time double-buffered inference with CMSIS-NN/CMSIS-DSP acceleration, actuating an on-board LED as a placeholder for the dispenser output.
Expected Technical Output	Logic-analyzer-verified SAI1/GPDMA1 audio driver; C++/C mixed-compilation wrapper (kws_inference.hpp/.cpp) bridging main.c to the Edge Impulse SDK; a double-buffered real-time inference loop; LED (green/red) actuation matching the spoken “yes”/“no” label; a documented open-issue list carried into Week 2.
Expected Evidence Type	Logic analyzer capture (SCK/WS timing), UART test log (ei_printf classification output), source code (kws_inference.cpp/.hpp, main.c), run_classifier() timing measurement in milliseconds.
Relation to Final Book Chapter	Chapter 2 — I2S/DMA Audio Acquisition Architecture for Edge AI; Chapter 3 — Edge Impulse / CMSIS-NN Real-Time Inference Pipeline on Cortex-M33.
Success Criteria	Audio acquisition free of the previous noise/dead-mic symptom, with SCK/WS ratio consistent with the theoretical 32-bit × 2-channel SAI framing (≈64); static classifier test reproduces the Edge Impulse Studio confidence value within expected tolerance (yes: 0.91016); real-time inference toggles the correct LED per spoken word with zero double-buffer overrun events.

SECTION 2 ENGINEERING QUESTION

This Week's Core Engineering Question:

Given a measured Edge Impulse run_classifier() duration of ≈450 ms against a ≈1000 ms single-window audio accumulation period on the STM32U5A9J-DK (Cortex-M33, CMSIS-NN/CMSIS-DSP accelerated), what buffering architecture is required to guarantee zero audio loss during inference — and is a simpler double-buffer swap technically sufficient compared to Edge Impulse's sliding-window run_classifier_continuous() API? Separately, does the ≈1.25% microphone sample-rate deviation (16.2 kHz measured vs. 16 kHz trained) introduce a measurable degradation in keyword-classification confidence?

SECTION 2A LITERATURE REFERENCE LOG [NEW — BOOK BIBLIOGRAPHY BUILDER]

#	Full IEEE Citation	Key Insight Relevant to This Week	Section of Book It Supports
1	L. Lai, N. Suda, and V. Chandra, "CMSIS-NN: Efficient Neural Network Kernels for Arm Cortex-M CPUs," arXiv:1801.06601, 2018.	Reports 2.6–5.4× runtime improvement of CMSIS-NN's quantized (INT8/q7) kernels over baseline CMSIS-DSP convolution on Cortex-M cores. This directly explains why this week's confirmed call path Register_CONV_2D() → arm_convolve_wrapper_s8() (verified in conv.cpp) is the performance-critical design choice for meeting real-time inference latency.	Chapter 3 — CMSIS-NN Acceleration Verification
2	Y. Zhang, N. Suda, L. Lai, and V. Chandra, "Hello Edge: Keyword Spotting on Microcontrollers," arXiv:1711.07128, 2017.	Demonstrates that 8-bit post-training quantization of small CNN/DNN KWS models incurs negligible accuracy loss versus full-precision baselines, supporting the validity of the fully INT8-quantized 2-conv-layer CNN architecture reverse-engineered this week from tflite_learn_1046205_7_compiled.cpp.	Chapter 3 — Model Architecture Analysis

SECTION 3 PLANNED vs COMPLETED ACTIVITIES

No.	Planned Activity (WBS Reference)	Completed Output	Status	Evidence File Name / Commit Hash
1	WBS 2.2 — Perancangan Pipeline Pre-processing (DSP)	Confirmed the DSP/MFCC preprocessing path is CMSIS-DSP-backed via macro inspection (EIDSP_USE_CMSIS_DSP) and compiler flags	Done	-
2	WBS 3.1 — Setup & Verifikasi Perangkat Keras I2S + GPDMA1	SAI1 Master Receive (32-bit) + GPDMA1 circular linked-list DMA configured; mic re-powered from external 3V3 (ESP32) after STM32 3V3 rail root-caused as faulty	Done	
3	WBS 3.2 — Uji Akuisisi Audio Mentah (Verifikasi Buffer)	Logic analyzer confirmed SCK = 1.04 MHz, WS = 16.2 kHz, ratio ≈ 64.2 (consistent with 32-bit × 2-channel framing)	Done	01_AkuisisiAudio
4	WBS 3.3 — Integrasi Pustaka CMSIS-DSP untuk Cortex-M33	CMSIS-NN/CMSIS-DSP macros confirmed active; FPU hard-float compiler flags confirmed; arm_convolve_wrapper_s8() call path confirmed in conv.cpp	Done	-
5	WBS 4.1 — Akuisisi Dataset Publik (KWS Basic)	Edge Impulse public “Yes-No” CMSIS-Pack model (4 labels) selected and integrated, in place of self-collected dataset	Done (ahead of schedule)	-
6	WBS 4.2 — Ekstraksi Fitur & Training Model Baseline	Used pre-trained public baseline rather than training in-house; static + real-time inference validated instead	Done (ahead of schedule)	02_KWS_Simple

Completion Summary:

Total Planned	Total Done	In Progress	Not Started	Completion Rate (%)
4 (WBS 2.2, 3.1–3.3)	4, +2 bonus (WBS 4.1–4.2)	0	0	100 (+2 pulled forward)

SECTION 4 KNOWLEDGE GAINED

Concept Learned	Your Own Explanation (minimum 3 sentences per concept)
Key Concept 1: DMA Double-Buffering vs. Blind-Spot Elimination	In the earlier single-buffer design, once the 16000-sample window was full, any audio arriving while run_classifier() was still executing (≈450 ms) was silently dropped rather than delayed. This created a recurring ≈450 ms “blind spot” per ≈1-second cycle. Switching to two alternating buffers means the ISR always has one buffer free to fill while the other is being classified, so accumulation never stops. This matters because for a voice command interface, losing the tail or head of a spoken word is functionally equivalent to not hearing it at all.
Key Concept 2: CMSIS-NN SIMD Kernel Dispatch vs. Generic Reference Path	TFLite Micro's operator registration (e.g. Register_CONV_2D()) branches at compile time between a CMSIS-NN-accelerated implementation and a generic C reference implementation, controlled by the EI_CLASSIFIER_TFLITE_ENABLE_CMSIS_NN macro. On this project, that branch was traced directly in conv.cpp and confirmed to resolve to arm_convolve_wrapper_s8(), which uses INT8 SIMD instructions native to the Cortex-M33's ARMv8-M DSP extensions. This matters because it is the concrete, code-level proof — not an assumption from the toolchain settings — that the “hardware acceleration” claimed for this board is actually being exercised by this specific model.
Key Concept 3: The signal_t / get_data Callback Abstraction	The Edge Impulse SDK does not accept a raw float pointer for input audio; instead it takes a signal_t struct containing a total_length and a get_data(offset, length, out_ptr) callback, which the SDK calls itself to pull samples as needed. This indirection lets the same SDK code work whether all samples are already in RAM (our case) or need to be streamed in slices (run_classifier_continuous()). Understanding this pattern was necessary to correctly explain why our own get_signal_data() function in kws_inference.cpp is just a bounded memcpy from a static float buffer, not a data source in its own right.
Misconception Corrected	I previously concluded the INMP441 microphone itself was physically damaged, based on the SD line showing no toggling while SCK/WS looked normal. I now understand the actual root cause was an unreliable 3V3 supply rail from the STM32 board, not the sensor — powering the mic from an external 3V3 source (from an unused ESP32 board, used purely as a power rail) immediately resolved it. This reinforces that a dead-looking digital line should first be checked against its power rail before the peripheral itself is assumed faulty.
Connection to Prior Knowledge	This week's SAI1/GPDMA1 timing work is a direct continuation of the Week 0 UART/clock-tree exercise: both ultimately depend on the STM32U5's PLL configuration being correct, and both were diagnosed the same way — by measuring the actual signal (baud rate / SCK-WS ratio) rather than trusting the theoretical .ioc calculation. The Week 0 lesson that “clock mismatches manifest as corrupted or absent data, not obvious errors” applied directly to this week's SAI debugging.

SECTION 5 DESIGN DECISION RECORD

#	Decision Made	Alternatives Considered	Reason for Selection (technical justification)	Risk / Limitation / When to Revisit
1	Power the INMP441 mic from an external 3V3 rail (unused ESP32 board, power-only) instead of the STM32's own 3V3	A: Keep using STM32 3V3 and continue debugging the rail. B: Replace the microphone hardware, assuming it was damaged.	Logic-analyzer evidence showed SCK/WS toggling normally while SD stayed flat — consistent with an insufficient/unstable supply, not a dead sensor. Swapping to an external, known-good 3V3 source isolated and confirmed the root cause quickly.	Depends on an external board purely as a power source, which is not a production-viable solution. Must be revisited when moving to a standalone dispenser PCB — likely needs a dedicated LDO or STM32 power-rail fix instead.
2	Use a double-buffer (ping-pong) accumulation scheme instead of Edge Impulse's run_classifier_continuous ()	A: run_classifier_continuous () — sliding-window/slice-based inference, Edge Impulse's built-in solution for this exact problem. B: Double-buffer — swap between two full-size window buffers.	Double-buffer is simpler to reason about and verify (a single overrun counter proves correctness), and is proportional to the problem: it only needed to guarantee non-stop accumulation, not reduce latency. run_classifier_continuous () requires additional SDK parameters (slice size) not yet understood in depth.	Double-buffer uses roughly 2× the RAM of a single window and does not reduce the ≈450 ms run_classifier() duration itself. If RAM becomes constrained, or the multi-word dispenser command model needs lower latency, run_classifier_continuous () should be reconsidered.
3	Defer implementing a confidence threshold; keep pure argmax label selection for this week	A: Add a minimum-confidence threshold now, blocking LED actuation on ambiguous (near-uniform) classifications. B: Ship pure argmax now, and treat thresholding as an explicit next-step once real usage data exists.	No real distribution of confidence values under silence/ambient-noise conditions had been collected yet; picking a threshold number without that data would be a guess dressed up as an engineering decision.	Currently, any argmax winner — even at ~25% confidence in a near-uniform 4-way split — will toggle the LED. This is an accepted, temporary risk and is the top carry-forward item for Week 2 (see Section 12).

SECTION 6 TECHNICAL RESULTS

6.1 Performance Metric Table

Metric	Specification / Target	Measured Result	Pass / Fail	Evidence File / Figure Reference
I2S SCK / WS ratio	≈ 64 (32-bit × 2 channel, theoretical)	SCK = 1.04 MHz, WS = 16.2 kHz → ratio ≈ 64.2	Pass	Logic analyzer capture
Audio sample rate	16.000 kHz (model training rate)	16.2 kHz actual (≈1.25% deviation)	Flagged — not yet evaluated for accuracy impact	Logic analyzer capture
Static classifier confidence (“yes” sample)	Dominant label = “yes”, high confidence	yes: 0.91016	Pass	UART log (ei_printf)
run_classifier() duration	No fixed spec this week; used to size the buffer scheme	≈450 ms, consistent across runs	Info / used as design input	UART log (timing print)
Double-buffer overrun count	0 (must remain 0 for design assumption to hold)	0, observed across all real-time tests so far	Pass	kwsDoubleBufferOverrunCount log
Real-time LED response to spoken word	LED matches spoken “yes”/“no”	Confirmed matching in direct user testing	Pass	Direct observation / video (TBD)

6.2 Reproducibility Note [NEW]

Result	Reproduction Conditions (tool version, settings, temperature, supply voltage, seed, etc.)
All results above	STM32CubeIDE v1.16, STM32CubeU5 Firmware Package v1.3.0 (same toolchain baseline as Week 0 — unchanged this week; update this row in the next WER if the toolchain version changes). Board powered via ST-LINK V3E. UART: USART1, 921600 baud, 8N1, no parity, via ST-LINK Virtual COM Port. SAI1 Master Receive, 32-bit data size, GPDMA1 circular linked-list mode. Microphone powered from an external 3V3 rail (ESP32 board used as a power source only).

6.3 Result Figures / Waveforms / Screenshots

		Implementasi Voice User Interface (VUI) dan Pemrosesan Audio Digita Menggunakan ARM Cortex-M33 pada Ekosistem STM32U5							
ID	Task / Deliverable	W0	W1	W2	W3	W4	W5	W6	W
1. REQUIREMENT & STUDY									
1.1	Riset Fundamental Voice Activation & Ekosistem STM32								
1.2	IDE & Environment Setup								
1.3	Analisis Keterkaitan AI & DSP								
1.4	Eksekusi Program Sederhana (Verifikasi Board)								
2. SYSTEM ARCHITECTURE & DESIGN									
2.1	Desain Arsitektur Data Flow I2S								
2.2	Perancangan Pipeline Pre-processing (DSP)								
3. PLATFORM BRING-UP & ACQUISITION									
3.1	Setup & Verifikasi Perangkat Keras I2S + GPDMA1								
3.2	Uji Akuisisi Audio Mentah (Verifikasi Buffer)								
3.3	Integrasi Pustaka CMSIS-DSP untuk Cortex-M33								
4. BASELINE AI MODELING (EDGE IMPULSE)									
4.1	Akuisisi Dataset Publik (KWS Basic)								
4.2	Ekstraksi Fitur & Training Model Baseline								
4.3	Verifikasi Model Baseline di Hardware								
5. CUSTOM TRIGGER WORD DEVELOPMENT									
5.1	Perekaman Dataset Trigger Word Utama								
5.2	Training & Tuning Parameter Trigger Word								
5.3	Uji Coba Klasifikasi Trigger Word secara Real-Time								
6. EKSPANSI COMMAND & INTEGRASI SISTEM									

Figure 6.1: WBS Gantt-chart execution status through Week 1. Teal cells mark planned weeks. Items 2.2 and 3.1–3.3 (Week 1, audio acquisition) and items 4.1–4.2 (Week 2, KWS baseline) were both completed within this reporting period.

Figure 6.2: Logic analyzer I2S

```
no: 0.07031
noise: 0.00000
unknown: 0.01953
yes: 0.91016
[REALTIME] KWS: run_classifier took 462 ms
```

Figure 6.3: KWS Result

SECTION 7 PROBLEMS ENCOUNTERED AND ROOT CAUSE ANALYSIS

#	Problem Description	Impact	Root Cause	Corrective Action	Verified Fixed?
1	INMP441 SD pin showed no toggling at all, while SCK/WS looked normal on the logic analyzer	Audio acquisition blocked; initially misdiagnosed as dead hardware	3V3 supply rail from the STM32 board was unreliable / insufficient for the mic, not a mic fault	Powered the mic from an external 3V3 rail (unused ESP32 board, power-only); also moved SD from PE6 to PD6 in the same pass	Yes
2	“Conflicting linkage” compiler error on ei_printf when wiring the Edge Impulse SDK into the C++ wrapper	Blocked compilation of kws_inference.cpp	ei_printf was declared with extern "C" while the SDK's own declaration in ei_run_dsp.h uses ordinary C++ linkage	Removed extern "C" from the ei_printf definition, keeping standard C++ linkage	Yes
3	UART terminal output appeared “staircased” (each new line indented further than the last)	Log output unreadable	Log strings used \n only; the terminal needed \r\n	Standardized every log string in the KWS wrapper to \r\n	Yes
4	Confidence values (%.5f) did not appear in the UART output at all	Could not verify classification confidence numerically	newlib-nano (the default C library) strips float support from printf by default	Added -u _printf_float to the linker's “Other flags” in Project Properties	Yes
5	UART output became corrupted (garbled/box characters) when raw audio streaming and KWS logging ran in the same window	Could not debug audio and KWS output simultaneously	ExtractAndSendHalfBuffer used HAL_UART_Transmit_DMA while ei_printf used blocking HAL_UART_Transmit, both on the same huart1 handle at the same time	Disabled raw audio streaming (wrapped in #if 0); must be re-enabled only with KWS logging turned off	Yes (workaround)
6	Single-buffer accumulation design lost ≈450 ms of audio every inference cycle while run_classifier() was executing	Words spoken during that window were partially or fully missed by the classifier	run_classifier() (≈450 ms) blocked new accumulation into the same buffer that was already full and awaiting processing	Redesigned to a double-buffer scheme: ISR always fills the buffer not currently being classified, with an overrun counter to detect if this assumption is ever violated	Yes

SECTION 8 VERIFICATION CHECKLIST

Verification Item	Status	Evidence / Comment
Requirement from WBS has been checked against this week's output	☑ Yes	Section 3, WBS 2.2 / 3.1–3.3 / 4.1–4.2
Architecture or block diagram has been updated to reflect this week's decisions	☑ Yes	Firmware stack diagram, Section 9.3
Simulation / code / test has been executed and output is non-trivial	☑ Yes	UART log yes: 0.91016; real-time LED test
All results are saved, version-controlled, and traceable to a repository link	☑ Yes	See Repository / Folder Link in Section DC
Every failure or limitation has been documented in Section 7	☑ Yes	6 problems logged
Figures have descriptive captions with units and conditions	☑ Yes	Figures 6.1–6.3
At least 2 literature references logged in Section 2A	☑ Yes	Lai et al. 2018; Zhang et al. 2017
Next week's plan is defined and linked to WBS	☑ Yes	
Section 13 (Historical Note) has been written for future cohort	☑ Yes	
Section 15 (Chapter Contribution Statement) has been written	☑ Yes	

SECTION 9 THEME-SPECIFIC ENGINEERING ADDENDUM

Perancangan Sistem Kontrol Dispenser Berbasis Perintah Suara Pada STM32U5A9J-DK: Studi Kinerja Dsp Extension Arm Cortex-M33 Dalam Pemrosesan Audio

DSP / VHDL Parameter / Item	This Week's Entry — Specific Values and Context
Board / chipset used	STM32U5A9J-DK (STM32U5A9NJH6Q, Arm Cortex-M33)
Peripheral(s) worked this week	SAI1 (I2S Master Receive, 32-bit) + GPDMA1 (circular, linked-list); USART1 (921600 baud, ST-LINK VCP); GPIOE (LED_GREEN/LED_RED)
Firmware layer worked	HAL peripheral drivers (SAI/GPDMA/UART/GPIO); C++/C mixed-compilation bridge (extern "C"); Edge Impulse SDK integration layer (classifier + CMSIS-NN/CMSIS-DSP)
Communication method and config	USART1 at 921600 baud, 8N1, no parity, via ST-LINK Virtual COM Port. Raw audio streaming (DMA) currently disabled to avoid conflicting with blocking KWS log transmission on the same UART handle.
Firmware stack layer diagram	main.c (double-buffer accumulation, LED control) → kws_inference.cpp (C++ wrapper, extern "C" bridge) → Edge Impulse SDK (run_classifier, DSP + CMSIS-NN kernels) → HAL (SAI1 / GPDMA1 / USART1 / GPIO)
Measured result(s)	run_classifier() ≈450 ms; static test confidence yes: 0.91016; SCK = 1.04 MHz / WS = 16.2 kHz; double-buffer overrun count = 0; LED response confirmed matching spoken word
Key firmware insight (this week)	The most consequential insight was that a shared UART peripheral cannot mix a DMA-based transmit with a blocking transmit without corruption — this is not obvious from the HAL API surface alone and only appeared once both audio streaming and KWS logging were active simultaneously. The double-buffer redesign also clarified that “eliminating blind spots” and “reducing run_classifier() latency” are two separate problems: double-buffering solves the former without touching the latter at all, which is why the ~450 ms duration is still logged as an open metric rather than a fixed one. Finally, tracing the SDK's signal_t/get_data pattern showed that the abstraction exists specifically to let the same run_classifier() code serve both fully-buffered and streaming use cases.
Known limitation / open issue	No confidence threshold yet (pure argmax); ≈1.25% microphone sample-rate deviation (16.2 kHz vs. 16 kHz trained) not yet evaluated for accuracy impact; run_classifier() duration (≈450 ms) not yet reduced, only its impact mitigated.
Board / chipset used	STM32U5A9J-DK (STM32U5A9NJH6Q, Arm Cortex-M33)
Peripheral(s) worked this week	SAI1 (I2S Master Receive, 32-bit) + GPDMA1 (circular, linked-list); USART1 (921600 baud, ST-LINK VCP); GPIOE (LED_GREEN/LED_RED)

SECTION 10 AI-ASSISTED ENGINEERING LOG

#	AI Tool Used	Task Given to AI (brief description of prompt)	AI Output Summary	Used In Deliverable?	Human Verification Performed

SECTION 11 LESSONS LEARNED

#	Lesson Learned — Write as a reusable engineering guideline with context, action, and reason
1	When a digital sensor line appears completely dead (no toggling) while its clock/word-select lines look normal: check the sensor's supply rail with the logic analyzer / multimeter before assuming the sensor is damaged. This project lost significant time on a “dead mic” diagnosis that was actually a 3V3 rail issue on the MCU side.
2	Never mix HAL_UART_Transmit_DMA and HAL_UART_Transmit (blocking) on the same UART handle if they can run concurrently: the two transmission modes conflict at the peripheral level and corrupt output for both. If two independent log streams are needed on one UART, either serialize them explicitly or move one to a different UART instance.
3	Before committing to a double-buffer (or any fixed-size ping-pong) design for a periodic pipeline stage, measure the worst-case processing time of the slow stage and compare it against the accumulation period of the fast stage. The double-buffer assumption here only holds because run_classifier() (≈450 ms) is reliably shorter than one window's accumulation time (≈1000 ms); this must be re-verified if the model or window size changes.

SECTION 12 PLAN FOR NEXT WEEK

#	Next Task Description	WBS Reference	Priority	Expected Evidence / Deliverable
1	Implement a minimum-confidence threshold before acting on a classification result, using confidence values gathered from real silence/ambient-noise recordings to pick a defensible number	CF (carried forward, Isu A)	High	Updated KWS_RunInference() returning confidence alongside label index; documented threshold value with supporting data
2	WBS 4.3 — Verify the “Yes-No” baseline model's real-time behavior across a wider range of speakers/distances/noise conditions, ahead of full custom trigger-word work in Week 3	WBS 4.3	Medium	Test log covering multiple speakers and conditions
3	Begin scoping the dataset collection plan for the actual dispenser command words (e.g. “isi air panas”, “stop”), including recording the dispenser's own background/pump noise as a noise-class sample	WBS 5.1 (early prep)	Medium	Draft recording plan / word list document

Estimated hours for next week:

Task #	Estimated Hours	Reasoning / Assumptions
1		
2		
3		
Total	…	Maximum available hours this week: XX

SECTION 13 HISTORICAL NOTE FOR NEXT COHORT [MANDATORY]

Historical Note Category	Your Entry — Write for the next student who will do the same WBS task
If you were starting this WBS task today, what would you do first (before anything else)?	Before touching any Edge Impulse code, verify the microphone's power rail with a multimeter or logic analyzer power probe — do this even if the datasheet says the MCU's 3V3 pin should be sufficient. On this board, it wasn't, and that single check would have saved days of assuming the sensor was defective.
What was the most time-consuming problem you encountered?	Diagnosing the “dead” INMP441 SD line. SCK and WS looked perfectly normal on the logic analyzer, which strongly suggested a sensor fault rather than a power problem — the actual cause (STM32 3V3 rail) was only found after ruling out several code-level explanations first.
What mistake should the next student absolutely avoid?	Do not run HAL_UART_Transmit_DMA (for raw audio) and HAL_UART_Transmit blocking calls (for logging) on the same UART instance at the same time. This produced garbled output that looked like a baud-rate or framing problem, when the real issue was two incompatible transmission modes racing on one peripheral.
Which file, commit, code snippet, simulation script, or reference is the most useful starting point?	Start from kws_inference.hpp/.cpp (the C++/Edge Impulse wrapper) and the double-buffer accumulation logic in main.c — these are the verified, working real-time pipeline. Do not restart the wrapper from scratch; the extern "C" bridging and ei_printf override in this file already resolve several non-obvious linkage issues.
What would you do differently if you had to repeat this week from scratch?	I would check the microphone's supply rail on day one of audio bring-up, before spending time on DMA/SAI configuration debugging that turned out not to be the actual problem.
What advice do you have that was not in the WBS or any reference document?	When you eventually collect your own dispenser command-word dataset, do not record clean studio-quality audio only. Record the dispenser's own background noise (pump/compressor) for several minutes and include it as a background-noise class in Edge Impulse — otherwise the model that works perfectly in a quiet room may fail in front of the actual appliance.

SECTION 14 SUPERVISOR REVIEW

Completed by the industrial mentor and/or academic supervisor after reviewing the WER. The student submits Rev 0.1; the supervisor annotates and returns it. The student incorporates feedback and resubmits as Rev 1.0.

Review Dimension	Supervisor Comment	Rating
Technical correctness	Are the measurements, calculations, and design decisions technically sound?	❑ Excellent ❑ Good ❑ Needs Work
Engineering reasoning	Is the student's explanation of cause-and-effect technically rigorous?	❑ Excellent ❑ Good ❑ Needs Work
Evidence quality	Are all results backed by traceable evidence files?	❑ Excellent ❑ Good ❑ Needs Work
Writing quality (for book)	Is the writing clear, precise, and at publishable quality?	❑ Excellent ❑ Good ❑ Needs Work
Literature references	Are the references relevant, correctly cited, and sufficient?	❑ Excellent ❑ Good ❑ Needs Work
Historical note quality	Is the historical note specific enough to be useful for the next cohort?	❑ Excellent ❑ Good ❑ Needs Work
Overall improvement from last week	Comment on growth trajectory	❑ Improved ❑ Same ❑ Declined

Field	Entry
Specific items to revise before Rev 1.0	…
Outstanding strength of this WER	…
Approval status	❑ Approved (Rev 1.0 accepted) ❑ Revise and resubmit ❑ Repeat test / simulation
Supervisor signature + date	…

SECTION 15 BOOK CHAPTER CONTRIBUTION STATEMENT [NEW — MANDATORY]

Book Chapter Contribution — Week 1 (Track C)

Relevant book chapter title / section: Chapter 2 — I2S/DMA Audio Acquisition Architecture; Chapter 3 — Edge AI Model Integration and Real-Time Inference on Cortex-M33

Contribution Statement:

This week establishes the first fully working, end-to-end acquisition-to-actuation pipeline for the voice-activated dispenser project: real audio captured over I2S/SAI1 with GPDMA1 on the STM32U5A9J-DK is classified in real time by a CMSIS-NN-accelerated, INT8-quantized keyword-spotting model, with the result driving a visible on-board actuator (LED). The chapter's central contribution is a resolved root-cause narrative for a common Edge AI bring-up failure — a microphone that appears electrically dead but is in fact starved by an unreliable MCU power rail — together with a proportionate, verifiable solution (double-buffering) to the equally common problem of a classifier whose inference time exceeds its input window.

Furthermore, this week's work is the first point in the project where the correlation between “AI” and “DSP” stops being a diagram and becomes a traceable execution path: the confirmed arm_convolve_wrapper_s8() call inside conv.cpp is direct, code-level evidence that this specific model exercises the Cortex-M33's hardware SIMD/MAC extensions, rather than an assumption inherited from the board's marketing specification. This establishes a reproducible verification method — not just a working demo — that future cohorts can reapply to any Edge Impulse model deployed on this hardware.

Cross-track integration note (complete if your work this week connects to another track):

Integration Aspect	Description
How does this week's audio-acquisition work enable the KWS/inference stage?	The double-buffered SAI1/GPDMA1 pipeline is the direct input source for run_classifier(); without a continuous, loss-free 16000-sample window, the classifier would receive truncated or corrupted audio regardless of model quality.
How does this week's KWS integration interface with the future actuator (dispenser) stage?	The LED toggle logic implemented this week (label index → GPIO action) is the exact interface point that will later be replaced with real pump/valve control — the label→action mapping pattern, not the LED itself, is the reusable contribution.
How does this week's work validate the overall system?	By validating real, live-mic, real-time inference (not just a static offline test) with a visible, immediate output, this week proves the full sense→think→act loop is technically sound on this hardware before committing further effort to training a project-specific command model.